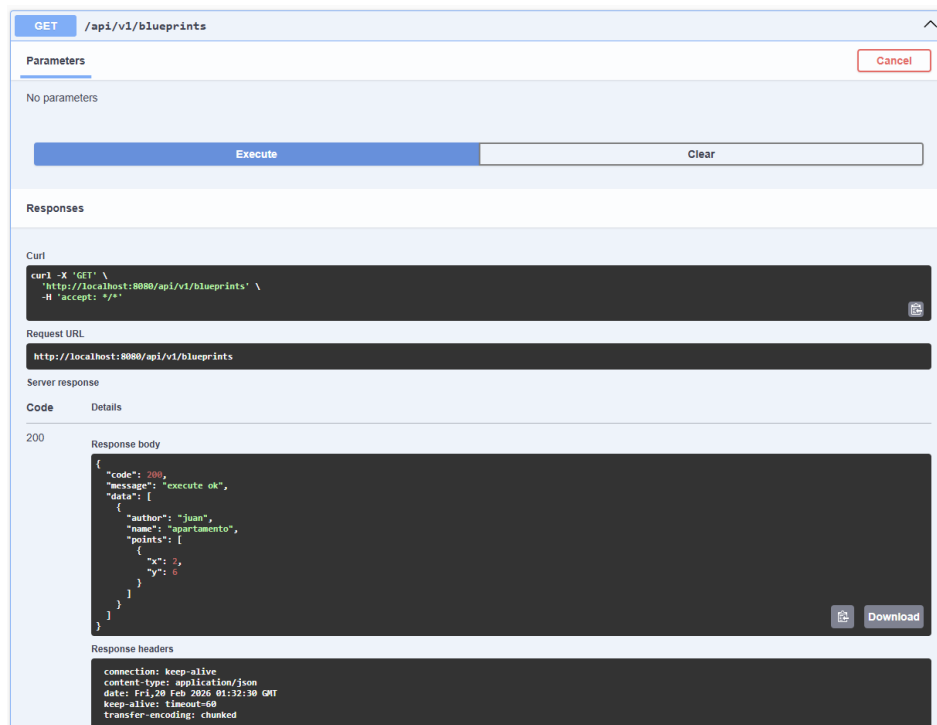


3. Buenas prácticas de API REST

- Cambia el path base de los controladores a /api/v1/blueprints.



- Usa **códigos HTTP** correctos:
 - 200 OK (consultas exitosas).



Consulta correcta.

- 201 Created (creación).

The screenshot displays a REST client interface with the following sections:

- POST /api/v1/blueprints**: The endpoint and method.
- Parameters**: A section with "No parameters" and "Cancel" and "Reset" buttons.
- Request body**: A text area containing a JSON object:

```
{  "author": "Robinson",  "name": "cocina",  "points": [    {      "x": 4,      "y": 10    }  ]}
```

 A dropdown menu shows "application/json".
- Execute**: A blue button to run the request.
- Clear**: A button to clear the request.
- Responses**: A section containing:
 - Curl**: A code block with the equivalent curl command:

```
curl -X 'POST' \  'http://localhost:8080/api/v1/blueprints' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "author": "Robinson",  "name": "cocina",  "points": [    {      "x": 4,      "y": 10    }  ]}'
```
 - Request URL**: A text field showing "http://localhost:8080/api/v1/blueprints".
 - Server response**: A section with a tabbed interface showing:
 - Code**: The status "201" and "Unauthenticated".
 - Details**: The response body, which is a JSON object:

```
{  "code": 201,  "message": "Blueprint created",  "data": {    "author": "Robinson",    "name": "cocina",    "points": [      {        "x": 4,        "y": 10      }    ]  }}
```

 A "Download" button is visible.
 - Response headers**: A section currently empty.

Creación blueprint exitosa.

- 202 Accepted (actualizaciones).

blueprints-api-controller

PUT /api/v1/blueprints/{author}/{bpname}/points

Parameters

Cancel Reset

Name	Description
author * required string (path)	Robinson
bpname * required string (path)	cocina

Request body required

application/json

```
{
  "x": 16,
  "y": 16
}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  http://localhost:8080/api/v1/blueprints/Robinson/cocina/points' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "x": 16,
    "y": 16
  }'
```

Request URL

http://localhost:8080/api/v1/blueprints/Robinson/cocina/points

Server response

Code	Details
202 undocumented	Response body <pre>{ "code": 202, "message": "Point added", "data": null }</pre>

Download

Actualización de datos exitosa.

- 400 Bad Request (datos inválidos).

POST

/api/v1/blueprints

Parameters

No parameters

Request body required

application/json

```
{  "author": "julian",  "name": "patio",  "points": [    {      "x": "hola",      "y": "hola"    }  ]}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  'http://localhost:8080/api/v1/blueprints' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "author": "julian",  "name": "patio",  "points": [    {      "x": "hola",      "y": "hola"    }  ]}'
```

Request URL

http://localhost:8080/api/v1/blueprints

Server response

Code	Details
400	Error: response status is 400

Undocumented

Response body

```
{  "timestamp": "2026-02-20T01:37:02.937+00:00",  "status": 400,  "error": "Bad Request",  "path": "/api/v1/blueprints"}
```

Download

Response headers

```
connection: close  content-type: application/json  date: Fri, 20 Feb 2026 01:37:02 GMT  transfer-encoding: chunked
```

Responses

Tipo de dato inválido error.

- 404 Not Found (recurso inexistente).

The screenshot shows a REST client interface with the following sections:

- Parameters:** A table with columns 'Name' and 'Description'. It contains one parameter:

Name	Description
author * required string (path)	carlos
- Execute:** A blue button labeled 'Execute' and a 'Clear' button.
- Responses:**
 - Curl:**

```
curl -X 'GET' \
'http://localhost:8080/api/v1/blueprints/carlos' \
-H 'accept: */*'
```
 - Request URL:** `http://localhost:8080/api/v1/blueprints/carlos`
 - Server response:**

Code	Details
404 <i>Undocumented</i>	Error: response status is 404

 - Response body:**

```
{
  "error": "Author not found"
}
```
 - Response headers:**

```
connection: keep-alive
content-type: application/json
date: Fri, 20 Feb 2026 01:37:40 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```
- Responses (Table):**

Code	Description	Links
200	OK	No links

Media type:
Controls Accept header.
Example Value | Schema:

```
{}
```

Autor no encontrado no existe en la base de datos.

- Implementa una clase genérica de respuesta uniforme:

```
J BlueprintsApiController.java 3, M    J BlueprintsServices.java    J ApiResponse.java U X
src > main > java > edu > eci > arsw > blueprints > dto > J ApiResponse.java > ApiResponse<T>
1  package edu.eci.arsw.blueprints.dto;
2
3  public record ApiResponse<T>{
4      int code,
5      String message,
6      T data
7  }
```

Con esta clase se puede ver respuestas más consistentes o sea mejor entendimiento y tenemos un buen manejo de errores.

```
import java.util.Map;
import java.util.Set;

@RestController
@RequestMapping("/api/v1/blueprints")
public class BlueprintsApiController {

    private final BlueprintsServices services;

    public BlueprintsApiController(BlueprintsServices services) { this.services = services; }

    // GET /blueprints
    @GetMapping
    public ResponseEntity<ApiResponse<Set<Blueprint>>> getAll() {
        Set<Blueprint> all = services.getAllBlueprints();
        return ResponseEntity.ok(new ApiResponse<>(200, "execute ok", all));
    }

    // GET /blueprints/{author}
    @GetMapping("/{author}")
    public ResponseEntity<?> byAuthor(@PathVariable String author) {
        try {
            return ResponseEntity.ok(services.getBlueprintsByAuthor(author));
        } catch (BlueprintNotFoundException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of(k1: "error", e.getMessage()));
        }
    }

    // GET /blueprints/{author}/{bpname}
    @GetMapping("/{author}/{bpname}")
    public ResponseEntity<ApiResponse<Blueprint>> byAuthorAndName(@PathVariable String author,
                                                                @PathVariable String bpname) {
        try {
            Blueprint bp = services.getBlueprint(author, bpname);
            return ResponseEntity.ok(new ApiResponse<>(200, "execute ok", bp));
        } catch (BlueprintNotFoundException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(new ApiResponse<>(404, e.getMessage(), null));
        }
    }

    // POST /blueprints
```